

-1 / +26 lines of 177 · 2 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (10 code line(s) added, 1 removed) · restructuring level 0

BEFORE	AFTER	file
132 } 133 134 static real_t kernel_s121(void) 135 { 136 int j;	132 } 133 134 static real_t kernel_s121(void) 135 { 136 int j; 137 /* Allocate temporary buffer to hold old values of a. 138 * This breaks loop-carried dependence by letting all iterations 139 * read from a constant copy instead of writing-then-reading in sequence. 140 real_t *a_old = (real_t*)malloc((size_t)LEN_1D * sizeof(real_t)); 141 if (!a_old) { 142 fprintf(stderr, "out of memory in kernel_s121\n"); 143 return (real_t)0; 144 } 145	
137 for (int nl = 0; nl < R; nl++) {	146 for (int nl = 0; nl < R; nl++) { 147 /* Copy current a to a_old for this iteration. 148 * j is the loop variable (automatically private). 149 * Loop is parallelizable since each thread writes to a different elem 150 #pragma omp parallel for 151 for (long i = 0; i < LEN_1D; i++) { 152 a_old[i] = a[i]; 153 } 154 155 /* Main computation loop: read from a_old (constant), write to a (inde 156 * j: private variable, each thread computes its own value (j = i+1). 157 * Loop is now parallelizable since iterations are independent: 158 * all read from the same a_old copy, write to different a[i] location 159 #pragma omp parallel for private(j) 160 for (int i = 0; i < LEN_1D-1; i++) { 161 j = i + 1; 162 a[i] = a_old[j] + b[i]; 163 } 164 165 pb_mix(nl); 166 }	
138 for (int i = 0; i < LEN_1D-1; i++) { 139 j = i + 1; 140 a[i] = a[j] + b[i]; 141 }	167 168 free(a_old); 169 return (real_t)0; 170 } 171 172 int main(int argc, char** argv) 173 {	
142 pb_mix(nl); 143 }		
144 return (real_t)0; 145 } 146 147 int main(int argc, char** argv) 148 {		